מטלת Kubernetes — Pods, ReplicaSets, Deployments, Services

Oct 29, 2026 · Almayo Mekonen

הוראות כלליות

המטלה בודקת שליטה בחמשת אבני היסוד של Kubernetes, ומשם ממשיכה לפרויקט קליל וכיפי שמחבר את כולן יחד.

חלק	מה עושים	זמן משוער
חלק א'	בדיקת שליטה: Pod, ReplicaSet, Deployment, ClusterIP, NodePort	כשעה וחצי
חלק ב'	פרויקט "מפעל עוגיות המזל"	כ-4 שעות

סביבת עבודה:

- קלאסטר מקומי: **minikube** (מומלץ). אפשר גם Docker Desktop עם Kubernetes מופעל.
- `kubectl` מותקן ומחובר לקלאסטר. בדיקה: `kubectl get nodes` מחזיר Node אחד במצב `Ready`.
- Docker ו-Node.js (לחלק ב').

כללים:

- כל משאב נוצר מקובץ YAML ונטען עם `kubectl apply -f`. אין ליצור משאבים עם `kubectl create` או `kubectl run`, חוץ ממקומות שמצוין במפורש.
- בכל תרגיל שכתוב בו "צלמו", מצרפים צילום מסך של הטרמינל לתיקיית `screenshots`.
- שאלות ההבנה נענות בקובץ `answers.md`, בתשובות קצרות ובמילים שלכם.

חלק א' — בדיקת שליטה

חמישה תרגילים, אחד לכל נושא. כל תרגיל בונה על הקודם, אז עושים אותם לפי הסדר.

תרגיל 1 — Pod

1. כתבו `pod.yaml` שמריץ Pod בשם `web-pod` עם האימג' `nginx:1.25`.
   - Labels: `app: web`, `tier: frontend`
   - `containerPort: 80`
   - `resources`: requests של `cpu: 100m`, `memory: 64Mi`, ו-limits של `cpu: 200m`, `memory: 128Mi`
2. הריצו `kubectl get pods -o wide` ו-`kubectl describe pod web-pod`. **צלמו.**

3. היכנסו לקונטיינר עם `kubectl exec -it web-pod -- sh` והריצו `curl localhost` (או `wget -qO- localhost`).
4. מחקו את ה-Pod. בדקו מה קרה.

שאלות:

- מה קרה אחרי המחיקה, ולמה?
- למה כמעט אף פעם לא מריצים Pod "ערום" בפרודקשן?
- מה ההבדל בין `requests` ל-`limits`?

תרגיל 2 — ReplicaSet

1. כתבו `replicaset.yaml` עם 3 רפליקות של `nginx:1.25`, ו-selector על `app: web-rs`.
2. פתחו טרמינל נוסף עם `kubectl get pods -w`. מחקו Pod אחד ותעדו מה קורה. **צלמו.**
3. **הניסוי המעניין:** צרו Pod נוסף ידנית (מקובץ YAML) עם label `app: web-rs`. מה קרה לו?
4. שנו בקובץ את האימג' ל-`nginx:1.27` והריצו `apply`. בדקו איזה אימג' רץ ב-Pods הקיימים: `kubectl describe pod <name> | grep Image`.

שאלות:

- מה קרה ל-Pod שיצרתם ידנית בסעיף 3? הסבירו את המנגנון.
- האם ה-Pods הקיימים עברו ל-`nginx:1.27`? מה זה אומר על השימוש ב-ReplicaSet לעדכוני גרסה?

בסוף התרגיל מחקו את ה-ReplicaSet.

תרגיל 3 — Deployment

1. כתבו `deployment.yaml` בשם `web` עם 3 רפליקות של `nginx:1.25`, label `app: web`.
    - `strategy`: `RollingUpdate` עם `maxSurge: 1` ו-`maxUnavailable: 0`
2. הריצו `kubectl get deploy,rs,pods`. **צלמו** וזהו את שרשרת הבעלות.
3. עדכנו את האימג' ל-`nginx:1.27` דרך הקובץ. עקבו עם `kubectl rollout status deployment/web`.
4. הריצו שוב `kubectl get rs`. כמה ReplicaSets יש עכשיו, ומה ה-replicas של כל אחד? **צלמו.**
5. **פריסה שבורה:** שנו את האימג' ל-`nginx:not-a-real-tag` והריצו `apply`. בדקו את מצב ה-Pods.
6. החזירו לגרסה התקינה עם `kubectl rollout undo deployment/web`. הציגו `kubectl rollout history deployment/web`. **צלמו.**

שאלות:

- תארו את הקשר Pod → ReplicaSet → Deployment.
- בסעיף 5, האם האתר הפסיק לעבוד? למה? איזו הגדרה אחראית לזה?
- למה ה-ReplicaSet הישן לא נמחק אחרי העדכון?

השאירו את ה-Deployment רץ — הוא משמש בתרגילים הבאים.

תרגיל 4 — Service מסוג ClusterIP

1. כתבו `svc-clusterip.yaml` בשם `web-svc` שמפנה ל-`app: web`, עם `port: 80` ו-`targetPort:` `80`.
2. בדקו גישה מתוך הקלאסטר (כאן מותר `kubectl run`):
   - `kubectl run tmp --rm -it --image=busybox -- sh`
   - בתוכו: `wget -qO- http://web-svc` וגם `wget -qO- http://web-svc.default.svc.cluster.local`
3. הריצו `kubectl get endpoints web-svc`. אחר כך הגדילו את ה-Deployment ל-5 רפליקות ובדקו שוב. **צלמו את שני המצבים.**
4. **שבירה מכוונת:** שנו את ה-selector של ה-Service ל-`app: wbe` (טעות כתיב). בדקו endpoints ונסו שוב `wget`. תקנו בחזרה.

שאלות:

- איך ה-Service יודע לאילו Pods לשלוח תעבורה?
- מה ראיתם ב-endpoints בסעיף 4, ולמה הבקשה נכשלה?
- האם אפשר לגשת ל-ClusterIP מהדפדפן במחשב שלכם? למה?

תרגיל 5 — Service מסוג NodePort

1. כתבו `svc-nodeport.yaml` בשם `web-nodeport` שמפנה ל-`app: web`, עם `nodePort: 30080`.
2. גשו לאתר מהדפדפן:
   - minikube: `minikube service web-nodeport --url`
   - Docker Desktop: `http://localhost:30080`
3. **צלמו** את דף ה-nginx בדפדפן.

שאלות:

- השלימו את הטבלה:

שדה	מה הוא מייצג	איפה הוא "חי"
`port`		
`targetPort`		
`nodePort`		

- מה טווח הפורטים המותר ל-NodePort?
- האם NodePort יוצר גם ClusterIP? איך בודקים את זה?
- מתי תבחרו ב-ClusterIP ומתי ב-NodePort?

חלק ב' — מפעל עוגיות המזל

בונים אפליקציה קטנה שמחלקת עוגיות מזל, ובכל עוגייה רואים **איזה Pod אפה אותה**. כך רואים בעיניים איך Kubernetes מחלק עומס, מחזיר Pods מתים לחיים ומעדכן גרסה בלי השבתה.

הזרימה: דפדפן ← NodePort ← Frontend (nginx) ← ClusterIP ← Fortune API (3 Pods)

שלב 1 — ה-API (כ-45 דקות)

1. צרו שרת Node.js + Express קטן עם endpoint אחד: `GET /api/fortune`.
2. התשובה בפורמט JSON:
   - `fortune` — משפט מזל אקראי מתוך רשימה של לפחות 10 (תהיו יצירתיים)
   - `pod` — שם ה-Pod, מתוך `os.hostname()`
   - `version` — מתוך משתנה סביבה `APP_VERSION`
3. כתבו Dockerfile ובנו אימג' `fortune-api:v1`.
   - minikube: הריצו קודם `eval $(minikube docker-env)` כדי שהאימג' ייבנה ישר בתוך הקלאסטר.
   - ב-YAML הגדירו `imagePullPolicy: Never` (האימג' מקומי ולא ב-Docker Hub).

שלב 2 — פריסת ה-API (כ-45 דקות)

1. כתבו Deployment בשם `fortune-api` עם 3 רפליקות ו-`APP_VERSION=v1` ב-`env`.
2. כתבו Service מסוג **ClusterIP** בשם `fortune-api` על פורט 3000.
3. בדקו מתוך Pod זמני שהבקשות ל-`http://fortune-api:3000/api/fortune` מחזירות שמות Pod שונים. **צלמו.**

שלב 3 — ה-Frontend (כשעה)

1. כתבו דף HTML עם כפתור "פתחו עוגייה". כל לחיצה קוראת ל-`/api/fortune` ומציגה:
   - את משפט המזל
   - את שם ה-Pod שענה ואת הגרסה
2. הגישו את הדף עם nginx, והוסיפו ל-config שלו proxy שמעביר את `/api/` ל-`http://fortune-api:3000`.
3. בנו אימג' `fortune-web:v1`, כתבו לו Deployment עם 2 רפליקות ו-Service מסוג **NodePort** על `30090`.
4. פתחו בדפדפן ולחצו כמה פעמים. **צלמו** תשובות משני Pods שונים לפחות.

שאלה: למה הדפדפן לא יכול לפנות ישר ל-`fortune-api`, ולמה דרך nginx זה עובד?

שלב 4 — כאוס במאפייה (כ-30 דקות)

בזמן שמישהו לוחץ על הכפתור בלי הפסקה:

1. מחקו Pod אחד של ה-API. האם האתר נפל? כמה זמן לקח עד שהוחזר Pod חדש?

2. הגדילו את ה-API ל-6 רפליקות. כמה שמות Pod שונים מופיעים עכשיו?
3. הקטינו ל-1. מה השתנה בחוויה?

תעדו כל ניסוי בשורה-שתיים ב-README. **צלמו** את `kubectl get pods -w` בזמן המחיקה.

שלב 5 — עוגיות v2 (כ-30 דקות)

1. שנו את רשימת המשפטים (או הוסיפו משהו מזהה, למשל קידומת "v2"), ובנו `fortune-api:v2`.
2. עדכנו את ה-Deployment ל-`v2` ו-`APP_VERSION=v2`, והריצו `apply`.
3. תוך כדי ה-rollout לחצו מהר על הכפתור. האם ראיתם תערובת של v1 ו-v2? **צלמו.**
4. החזירו ל-v1 עם `kubectl rollout undo` ובדקו בדפדפן.

שלב 6 — README (כ-15 דקות)

הסבר קצר על המבנה, פקודות הרצה מאפס, ותוצאות הניסויים משלבים 4 ו-5.

בונוסים

- מונה בדף שסופר כמה עוגיות אפה כל Pod.
- `readinessProbe` ל-API, והסבר מה הוא משנה בזמן rollout.
- כל הפרויקט ב-namespace נפרד בשם `bakery`.
- `resources` (requests ו-limits) לכל הקונטיינרים.

הגשה וניקוד

מגישים קישור לריפו ב-GitHub במבנה הזה:

```
k8s-assignment/
├── part-a/
│   ├── pod.yaml
│   ├── replicaset.yaml
│   ├── deployment.yaml
│   ├── svc-clusterip.yaml
│   ├── svc-nodeport.yaml
│   ├── answers.md
│   └── screenshots/
└── part-b/
    ├── api/          (קוד + Dockerfile)
    ├── frontend/     (HTML + nginx.conf + Dockerfile)
    ├── k8s/          (כל קבצי ה-YAML)
    ├── screenshots/
    └── README.md
```

רכיב	נקודות
תרגיל 1 — Pod	6
תרגיל 2 — ReplicaSet	8
תרגיל 3 — Deployment	10
תרגיל 4 — ClusterIP	8
תרגיל 5 — NodePort	8
שלבים 1–2 — API ופריסה	20
שלב 3 — Frontend ו-NodePort	20
שלבים 4–5 — כאוס ו-v2	15
שלב 6 — README	5
סה"כ	**100**
בונוסים	עד 10